

# Java Collections Framework — Interview Q&A

20 questions covering core interfaces, implementations, and internals

## Core Interfaces

### 1. What is the Java Collections Framework, and what are its core interfaces?

It's a unified architecture for storing and manipulating groups of objects, providing standard interfaces, implementations, and algorithms. The core interfaces are `Collection` (the root), with `List`, `Set`, and `Queue` extending it, plus `Map`, which sits separately since it stores key-value pairs rather than single elements.

### 2. What is the difference between `Collection` and `Collections`?

'`Collection`' is an interface representing a group of objects, the root of the collection hierarchy (`List`, `Set`, `Queue` extend it). '`Collections`' is a utility class containing static helper methods that operate on collections, such as `sort()`, `reverse()`, `unmodifiableList()`, and `synchronizedList()`.

### 3. What is the difference between `List`, `Set`, and `Queue`?

A `List` is an ordered collection allowing duplicates and indexed access. A `Set` disallows duplicate elements and generally has no guaranteed ordering (though `LinkedHashSet` and `TreeSet` provide ordering). A `Queue` is designed for holding elements prior to processing, typically in FIFO order, with specialized operations like `offer()`, `poll()`, and `peek()`.

### 4. What is the `Iterator` interface, and how does it differ from a for-each loop?

`Iterator` provides a standard way to traverse a collection one element at a time via `hasNext()` and `next()`, and crucially supports safe removal during iteration via `remove()`. A for-each loop is syntactic sugar over `Iterator` internally, but it does not allow safe structural modification of the collection during iteration — doing so throws a `ConcurrentModificationException`.

### 5. What is the difference between `Comparable` and `Comparator`?

`Comparable` is implemented by the class itself, defining a single natural ordering via `compareTo()` (e.g. `String`'s alphabetical order). `Comparator` is a separate class defining one or more custom orderings via `compare()`, passed externally to sorting methods — useful when you need multiple sort orders or can't modify the original class.

## List & Set Implementations

### 1. What is the difference between `ArrayList` and `LinkedList`?

`ArrayList` is backed by a dynamic array, giving  $O(1)$  index-based access but  $O(n)$  insertion/removal in the middle (due to shifting elements). `LinkedList` is backed by a doubly linked list, giving  $O(1)$  insertion/removal at the ends but  $O(n)$  index-based access (requiring traversal), and it also implements `Deque`.

### 2. What is the difference between `ArrayList` and `Vector`?

Both are backed by a dynamic array and behave similarly, but `Vector` is synchronized (thread-safe) at the method level, making it slower due to locking overhead, while `ArrayList` is not synchronized. `Vector` is considered legacy; for thread-safe needs, `Collections.synchronizedList()` or `CopyOnWriteArrayList`

are generally preferred today.

### **3. What is the difference between HashSet, LinkedHashSet, and TreeSet?**

HashSet offers  $O(1)$  average add/contains/remove via hashing but no guaranteed iteration order. LinkedHashSet maintains insertion order by additionally linking entries, at a slight memory/performance cost. TreeSet keeps elements sorted (natural order or via a Comparator), backed by a red-black tree, giving  $O(\log n)$  operations.

### **4. How does HashSet ensure that duplicate elements aren't added?**

When an element is added, HashSet computes its hashCode() to determine the bucket, then uses equals() to check whether an element considered equal already exists in that bucket. If both hashCode() and equals() indicate the element is a duplicate, the add is rejected.

### **5. What is the significance of overriding both hashCode() and equals() together for objects stored in a HashSet or as HashMap keys?**

The contract requires that equal objects (per equals()) must produce the same hashCode(). If you override equals() without overriding hashCode() consistently, two 'equal' objects could land in different buckets, causing a HashSet to store duplicates or a HashMap to fail to retrieve a value for a key that should match.

## **Map Implementations**

### **1. What is the difference between HashMap, LinkedHashMap, and TreeMap?**

HashMap offers  $O(1)$  average get/put with no guaranteed order. LinkedHashMap maintains insertion order (or optionally access order, useful for LRU caches) by linking entries alongside the hash table. TreeMap keeps entries sorted by key, backed by a red-black tree, giving  $O(\log n)$  operations but sorted iteration and range queries.

### **2. How does a HashMap handle collisions internally?**

When two keys hash to the same bucket, HashMap stores them together in that bucket, traditionally as a linked list traversed using equals() to find the correct entry. Since Java 8, if a bucket's linked list grows beyond a threshold (default 8), it's converted to a balanced tree (red-black tree) for that bucket, improving worst-case lookup from  $O(n)$  to  $O(\log n)$ .

### **3. What is the difference between HashMap and Hashtable?**

HashMap is not synchronized (not thread-safe by default) and allows one null key and multiple null values. Hashtable is synchronized at the method level (thread-safe but slower) and doesn't allow any null keys or values. Hashtable is a legacy class; ConcurrentHashMap is the modern thread-safe alternative.

### **4. What is ConcurrentHashMap, and how does it achieve thread safety more efficiently than a synchronized HashMap?**

ConcurrentHashMap achieves thread safety by locking only small segments/buckets of the map rather than the entire structure (in newer versions, using fine-grained locking per bin plus CAS operations), allowing multiple threads to read and write different parts concurrently — unlike Collections.synchronizedMap(), which locks the entire map for every operation.

## 5. What happens when you insert a key into a HashMap that already exists?

The new value replaces the old value associated with that key, and `put()` returns the previous value that was overwritten (or null if the key was new). The map's size doesn't increase since it's an update rather than an insertion.

## Behavior, Performance & Modern Usage

### 1. What is the load factor and initial capacity in a HashMap, and how do they affect performance?

Initial capacity is the number of buckets the HashMap starts with (default 16), and load factor (default 0.75) determines the fill ratio at which the map resizes (doubling capacity and rehashing all entries). A higher load factor saves memory but increases collision likelihood and lookup time; a lower one uses more memory but keeps lookups faster.

### 2. What is fail-fast versus fail-safe behavior in collection iterators?

Fail-fast iterators (used by `ArrayList`, `HashMap`, etc.) throw a `ConcurrentModificationException` if the collection is structurally modified while iterating, detected via an internal modification count. Fail-safe iterators (used by `CopyOnWriteArrayList`, `ConcurrentHashMap`) operate on a clone or a consistent snapshot of the data, so they don't throw this exception, though they may not reflect the very latest modifications.

### 3. What is the difference between `Collections.unmodifiableList()` and an immutable list created via `List.of()`?

`Collections.unmodifiableList()` wraps an existing mutable list in a read-only view — attempts to modify it throw `UnsupportedOperationException`, but the underlying list can still be changed through the original reference, which would be reflected in the view. `List.of()` (Java 9+) creates a genuinely immutable list with no underlying mutable backing list at all, and also disallows null elements.

### 4. How would you sort a List of custom objects by a specific field?

You can implement `Comparable` on the class to define a natural ordering, or (more flexibly) pass a `Comparator` to `Collections.sort()` or `list.sort()` — e.g. `list.sort(Comparator.comparing(Person::getAge))` sorts a list of `Person` objects by age without modifying the class itself.

### 5. What are the diamond operator and generics used for in the Collections Framework?

Generics (e.g. `List<String>`) let you specify the type of elements a collection holds at compile time, enabling compile-time type safety and eliminating the need for manual casting when retrieving elements. The diamond operator (`<>`) lets the compiler infer the generic type from the declared variable, so you can write `'new ArrayList<>()'` instead of repeating the full type.